What I decided, and what I left out

Kill the Quote Spreadsheet · https://kill-the-quote-spreadsheet.vercel.app

I picked IT hardware because it has the most unit traps of the categories I considered: kits, boxes of 10, boxes of 50, per-piece, per-DIMM. A unit trap is the dangerous kind of error, because the wrong number still looks like a sensible price. Nobody catches it by reading the screen.

Five decisions

1. The model reads. The code does the maths. The model only reports what a document says, in the supplier's own words and currency. Every calculation happens in ordinary code. *Why:* you cannot prove a model will add up the same way twice. You can prove code does. So I wrote the calculator twice, in two languages, and a test checks they agree on all 150 cells. *The pro:* the total cannot quietly drift between two runs, and every number on the screen can point at the sentence it came from.

2. Three answers to "did they qualify", not two. Passed, failed, and **nobody has checked yet**. *Why:* a supplier whose questionnaire nobody has opened is not a supplier who failed. Merging those two is how a real bid gets dropped by mistake. *The pro:* the screen can never accuse a supplier of failing something it has no evidence for, and the buyer can see exactly which suppliers still need checking.

3. Five kinds of blank, not one dash. A missing cell is one of five things: the supplier declined, never mentioned it, we could not read it, they wrote something that is not a price, or it needs the buyer's judgement. Each gets its own mark. *Why:* my first build drew all five the same way, which turned five different next actions into one shrug. *The pro:* the buyer knows what to do about each blank. Chase this supplier, ignore that one, look at this one yourself. A single dash tells you nothing.

4. When two totals are not comparable, say so before showing them. The strict award looks ₹45.3 lakh cheaper than the eligible-suppliers award. On the 28 lines they both cover, it is actually ₹5.0 lakh **more expensive**. The gap is coverage: one covers 28 lines, the other 30. *Why:* both numbers are correct, which is exactly why a careful person picks the wrong one. No amount of care catches this; only the system can. *The pro:* whenever two scenarios cover different lines, the buyer is told, and both totals are restated on the lines they share, before either headline appears.

5. The quality gate can be overridden, on purpose. The enquiry is held if a line says "box" without saying how many are inside. The buyer can send it anyway by typing a reason. *Why:* a hard block does not stop anyone. It just gets worked around by editing the enquiry until the warning disappears, and that leaves no record at all. *The pro:* the buyer stays in control, and the reason they typed prints in the award note. So the override is on the record, and they explain it to their CFO rather than to me.

Three things I got wrong

The analyst was answering one of my own demo questions from a lookup table. *What was wrong:* the tool that checked the questionnaire read a pass/fail flag and a hand-typed sentence out of a data file. So *"is Vector's ISO 27001 certificate actually valid?"* was answered by handing the model my own conclusion, which is exactly the shortcut this project rules out. *How I fixed it:* the verdict is now worked out from the supplier's actual answer and the certificate attached to it, every time it is asked.

The award recommendation did not add up. *What was wrong:* the tool gave the model thirty line-by-line amounts and no totals per supplier, so when asked to draft the award it added them up itself and got them wrong: ₹1.48 cr and ₹2.49 cr against real figures of ₹1.85 cr and ₹1.46 cr. Its three parts came to ₹4.70 cr against the ₹4.05 cr it had quoted correctly two sentences earlier. On the one sentence a buyer shows a CFO. *How I fixed it:* "the model must not do maths" is only a wish until the tool stops making it necessary. The tool now returns each supplier's total, worked out by the same code that worked out the headline, and the model quotes it.

The safety net on photographs caught nothing. This is the one I would lead with, because it is the central question: what does it show the buyer when it isn't sure?

What was wrong, in two layers. I read one rate card as five photographs of falling quality. The fourth came back 52% accurate while reporting 0.90 confidence on every wrong digit, so any check that fires on low confidence would never have fired. I switched the trigger to money: the eight largest numbers on any photograph get read a second time, independently, on a different model. Five of Vector's prices then disagreed badly. 84,000 against 99,500. 61,800 against 268. And **all five stayed in the total and won their line in the award.** The disagreement was recorded, shown on screen, and changed nothing, because it only lowered a confidence score and the calculator has never read confidence. The cache made it worse: it rebuilt every price from the model's raw answer on each reload, so even the confidence downgrade was thrown away. The first read of a document was safe and every read after it was not, which is the worst shape a bug can have, because it disappears the moment you go looking with a fresh file.

How I fixed it. A contested price is now excluded outright and put in front of a person, carrying both readings. The comparison moved the day I fixed it: cheapest-from-anyone went from ₹3.88 cr to ₹3.91 cr, which is the honest number. What is still open is that a smaller price, outside the largest eight, could be read wrong and look right. That is the top risk left in the build. I found all of this by fact-checking my own documentation, which claimed the price was pulled from the total. The claim was right about what should happen and wrong about what did.

What I left out, and why

- **Real email sending.** Stubbed by design, because the plumbing proves nothing and the reading proves everything. The channel choice is not faked: invite a supplier on WhatsApp and their certificate really does not arrive, because that is what the medium does.
- **Letting the buyer correct the system.** They can chase a supplier for a better answer, and every assumption is shown with its source and the alternative I rejected. But they cannot overwrite a price, and they cannot edit an assumption from the screen. Both are the right next features and both need an audit trail: an editable grid with no record of who changed what quietly destroys the one thing this product sells, which is that every number can say where it came from.
- **Multi-tenancy.** Supplier codes are unique across the whole system rather than per enquiry. Correct for one buyer, wrong for a real customer base. Not worth building before there is a second buyer.
- **Cutting the last ties to the shipped example.** You can draft your own enquiry and it works end to end, but it still borrows the example's buyer details and assumption list. Visible only to someone who drafts a second enquiry, so it lost to work every user will actually see.
- **Scale.** Tested at 30 lines and 5 replies, not 300 and 30. The scope was the size where it hurts, and that size is about judgement, not volume.
- **Pricing each ambiguity warning.** The gate tells the buyer what a supplier could do with a vague line. What that vagueness would cost in rupees is the better warning, and I ran out of time before building it.

The problem I think matters more

The obvious problem. Almost every mess in this data starts with a vague line in the enquiry. Ask for "32GB DDR5 RDIMM" without saying "as a matched kit of two 16GB sticks" and three suppliers price three different things. Better AI never fixes that, because all three of them answered the question you actually asked. So I built the gate that holds the enquiry until the line is clear.

The better problem. Building that, I noticed something bigger. Reading documents was never the bottleneck. **Incomplete replies are.**

Helios sent a five-line email. Two of thirty lines priced, no questionnaire, no terms. The 28 lines they ignored are worth **₹2.92 crore**. No model, however good, can read a price that was never written. That is ₹2.92 crore of the deal sitting behind a follow-up nobody sent.

So the system works out exactly what each supplier still owes, asks for only that with a deadline, and records the ask. And it keeps two things apart that look identical on a screen: *"we never got it"* and *"we asked on the 10th, gave them until the 16th, and nothing came back"*. Only the second one lets a buyer award the business to somebody else and defend that decision afterwards.

Get very good at reading documents and stay bad at closing the loop, and all you have built is a beautiful comparison of incomplete information. That is where I would spend the next week.